

UE5 C++ MASTERY ÇALIŞMA MERKEZİ · SUJET

04

BATTLE BLASTER

Pawn/GameMode mimarisiyle çalışan bir tank savaşı kurmak: mermi sistemi, composition tabanlı hasar mimarisi, HUD ve seviyeler arası kalıcı veri.

SÜRE

5 saat 58 dakika

DERS SAYISI

46 ders

ALİŞTIRMA

9 zorunlu + 1 bonus

XP

150 (+35 bonus)

BÖLÜM 0 — GENEL BİLGİLER VE KURALLAR

0.1 Önsöz

İlk kez gerçek bir “oyun döngüsü” kuruyorsun: başla, oyna, kazan ya da kaybet, yeniden başla. Bu bölüm önceki bölümde öğrendiğin composition mantığını (Health Component) ve pointer/delegate bilgini birleştirip, bunları bir GameMode/GameInstance mimarisi içinde bir araya getirmeni ister.

0.2 Amaç

Pawn/GameMode mimarisini kullanarak hareket eden bir tank, fareyle döndürülen bir kule, zamanlayıcıyla ateş eden düşman kuleler, composition tabanlı ortak bir hasar sistemi ve seviyeler arası veri taşıyan bir Game Instance ile eksiksiz bir oyun döngüsü kurmak.

0.3 Genel Yönlendirmeler

- Health Component'i yazarken “bunu hem oyuncu hem düşman kullanabilir mi?” diye sor.
- Timer kullan, Tick içinde manuel süre sayma — okunabilirlik ve performans için.
- Game Instance'a sadece gerçekten seviyeler arası kalıcı olması gereken veriyi koy.
- Her yeni mekaniği (mermi, hasar, UI) eklemekten önce bir öncekinin çalıştığını doğrula.

0.4 Değerlendirme Kriterleri (Kendi Kendine Savunma)

Bir 42 öğrencisinin projesini savunması gibi, aşağıdaki sorulara videoya veya nota bakmadan, kendi cümlelerinle cevap verebilmelisin:

- Game Mode ile Game Instance arasındaki yaşam döngüsü farkı nedir?
- Composition ile yazdığın Health Component'i üçüncü bir Actor türüne nasıl ekledin?
- AddActorLocalOffset ile SetActorLocation arasındaki fark nedir, neden tank hareketi için local'i seçtin?
- ApplyDamage/OnTakeAnyDamage sistemini kullanmanın, kendi hasar değişkenini elle azaltmaya göre avantajı nedir?

0.5 Zorunlu ve Yasak

✓ ZORUNLU

- Hasar sistemini composition ile ortak bir Health Component üzerinden kurmak
- Mermi ateşleme için Timer kullanmak (Tick ile manuel sayma değil)
- Seviyeler arası kalıcı veriyi Game Instance'ta tutmak

✗ YASAK

- Oyuncu ve düşman için ayrı ayrı, birbirinden bağımsız hasar mantığı yazmak
- Game Mode'da seviye değişince kaybolması gereken veriyi “kalıcı” veri olarak kullanmak
- Hasar uygulamasını Unreal'ın ApplyDamage sistemini bypass ederek elle yapmak

0.6 XP ve Rank Sistemi

Bu bölümün her alıştırması bir XP değeri taşır. Zorunlu kısmı eksiksiz bitirince **150 XP** kazanırsın; bonus dahil üst sınır **185 XP**'dir (bonus, ancak zorunlu %100 tamamlandıktan sonra

açılır). Kazandığın XP'yi Notion'daki “XP Takip Sistemi” veritabanında işaretleyerek genel kurs rankını görebilirsin — bu bölümü bonusuyla bitirince **Gameplay Mühendisi** rankına yaklaşırsın.

BÖLÜM I — ZORUNLU KISIM

Alıştırmalar birbiri üzerine inşa olacak şekilde sıralanmıştır — bir öncekini bitirmeden sonrakine geçme. Her alıştırma, ilgili Notion sayfasındaki “Konu Anlatımı” bölümünün karşılık gelen kısmına dayanır.

EXERCISE 00 · 15 XP

Tank Pawn

İlgili dersler: Base Pawn Class · Child C++ Class · Camera & Spring Arm · C++ Game Mode

Amaç — Bir Pawn taban sınıfından türeyen, Spring Arm ve Camera'ya sahip bir Tank sınıfı ve onu kontrol eden bir Game Mode kurmak.

► Yapılacaklar

- APawn'dan türeyen bir taban sınıf yaz, ondan bir Tank child sınıfı türet.
- Spring Arm ve Camera Component ekle.
- C++ Game Mode'da DefaultPawnClass'ı Tank olarak ayarla.

⚠ Kısıtlar

- Kamera mantığını doğrudan Tank sınıfına değil, Spring Arm üzerinden kur.

✓ Beklenen Sonuç

- Play'e basınca tank sahnede kontrol edilebilir durumda, kamera arkasında takip ediyor.

EXERCISE 01 · 15 XP

Hareket Girdisi

İlgili dersler: Input Mapping Context · Move Input · AddActorLocalOffset · AddActorLocalRotation

Amaç — Enhanced Input ile W/S/A/D tuşlarını tankın yerel ileri/geri hareketine ve dönüşüne bağlamak.

► Yapılacaklar

- Move eylemi için bir Input Action + Mapping Context oluştur.
- İleri/geri için AddActorLocalOffset, dönüş için AddActorLocalRotation kullan.
- Hareket hızını DeltaTime ile çarpmayı unutma.

⚠ Kısıtlar

- SetActorLocation ile dünya koordinatında hareket ettirme — tank kendi yönüne göre hareket etmeli.

✓ Beklenen Sonuç

- Tank hangi yöne bakıyorsa o yöne ilerliyor, yerinde dönebiliyor.

EXERCISE 02 · 15 XP

Kule Döndürme

İlgili dersler: Line Trace From Cursor · Rotating The Turret

Amaç — Fare imlecinin işaret ettiği 3D noktayı bulup tankın kulesini o noktaya doğru döndürmek.

► Yapılacaklar

- PlayerController::DeprojectMousePositionToWorld ile ekran konumundan 3D ışın çıkar.
- Bu ışınla bir line trace yap, çarpılan noktayı bul.
- Kuleyi (ayrı bir mesh/component) bu noktaya bakacak şekilde döndür.

⚠ Kısıtlar

- Trace başarısız olduğunda (hiçbir şeye çarpmazsa) kuleyi rastgele bir yöne döndürme — önceki yönde bırak.

✓ Beklenen Sonuç

- Kule, fare hareket ettikçe akıcı şekilde imleci takip ediyor.

EXERCISE 03 · 15 XP

Düşman Kuleleri

İlgili dersler: Enemy Tower Class · GetAllActorsOfClass · While/For Loop · Rotating Towards Player

Amaç — Sahnedeki tüm düşman kulelerini bulup her birini oyuncuya doğru döndüren bir sistem kurmak.

► Yapılacaklar

- Bir EnemyTower Actor sınıfı yaz.
- GetAllActorsOfClass ile sahnedeki tüm kuleleri bir TArray'de topla.
- Bir döngüyle her kuleyi oyuncunun anlık konumuna doğru döndür.

⚠ Kısıtlar

- GetAllActorsOfClass'ı her Tick'te çağırmak yerine sonucu makul bir sıklıkla (ör. Timer ile) güncellemeyi düşün — performans.

✓ Beklenen Sonuç

- Sahnedeki tüm kuleler, sayıları kaç olursa olsun, oyuncuya dönük duruyor.

EXERCISE 04 · 20 XP

Mermi Sistemi

İlgili dersler: Using Timers · Spawning Projectiles · Projectile Movement · This Keyword

Amaç — Kulelerin düzenli aralıklarla mermi ateşlemesini sağlayan bir Timer + SpawnActor sistemi kurmak.

► Yapılacaklar

- Bir Projectile Actor sınıfı yaz, Projectile Movement Component ekle.
- Kulede bir SetTimer ile düzenli aralıklarla ateş eden bir fonksiyon çağır.
- SpawnActor ile mermiyi namlu ucunda oluştur, this ile “beni kim ateşledi” bilgisini ver.

⚠ Kısıtlar

- Ateş sıklığını Tick içinde elle sayarak değil, Timer ile yönet.

✓ Beklenen Sonuç

- Kule, sabit aralıklarla oyuncuya doğru mermi fırlatıyor.

EXERCISE 05 · 25 XP

Hasar Sistemi

İlgili dersler: Health Component · Applying Damage · OnTakeAnyDamage · Handling Pawn Death · Super Function Call

Amaç — Hem oyuncu hem düşmanın kullanabileceği, composition tabanlı ortak bir Health Component yazmak.

► Yapılacaklar

- Bir UActorComponent alt sınıfı olarak HealthComponent yaz, CurrentHealth tut.
- OnTakeAnyDamage'e bağlanıp ApplyDamage ile gelen hasarı düş.
- Can sıfırlanınca bir “Öldü” delegate'i yayınla; her Actor kendi ölüm tepkisini (Super:: çağırısı dahil) buna bağlasın.

⚠ Kısıtlar

- Oyuncu ve düşman için ayrı hasar mantığı yazma — aynı component'i ikisine de ekle.

✓ Beklenen Sonuç

- Hem tank hem düşman kule, aynı component ile hasar alıp can bitince yok oluyor.

EXERCISE 06 · 15 XP

Oyun Sonu Akışı

İlgili dersler: Ternary Operatör · Restart Level

Amaç — Oyuncu öldüğünde ya da tüm düşmanlar yok edildiğinde oyunun bitmesini ve yeniden başlatılabilmesini sağlamak.

► Yapılacaklar

- Kazandın/kaybettin durumunu bir ternary operator ile ayarla.
- Fare imlecini görünür yap, oyunu duraklat/girdi kilitle.
- Bir tuşla OpenLevel çağırarak seviyeyi yeniden başlat.

⚠ Kısıtlar

- Oyun bittikten sonra oyuncunun hâlâ hareket edip ateş edebilmesini engellemeyi unutma.

✓ Beklenen Sonuç

- Oyun bitiminde ekranda net bir kazan/kaybet göstergesi var ve seviye yeniden başlatılabiliyor.

EXERCISE 07 · 15 XP

Game Instance ve Seviyeler

İlgili dersler: Game Instance · Level Management · Countdown Timer

Amaç — Seviyeler arasında kalıcı olması gereken veriyi (ör. mevcut seviye numarası) bir Game Instance'ta tutmak.

► Yapılacaklar

- Bir UGameInstance alt sınıfı yaz, mevcut seviye numarasını orada tut.
- Bir sonraki seviyeye geçiş fonksiyonu ekle.
- Seviye başında kısa bir geri sayım için Timer kullan.

⚠ Kısıtlar

- Seviye numarasını Game Mode'da tutma — her seviyede sıfırlanır.

✓ Beklenen Sonuç

- 3 farklı seviye arasında geçiş yapılırken hangi seviyede olduğu bilgisi kaybolmuyor.

EXERCISE 08 · 15 XP

HUD Widget'i

İlgili dersler: Widget Blueprint · User Widget Class · Creating The Widget

Amaç — Oyuncunun canını gösteren bir C++ User Widget yazıp ekrana eklemek.

► Yapılacaklar

- Bir UUserWidget alt sınıfı yaz, can değerini güncelleyecek bir fonksiyon ekle.
- UMG Designer'da görsel yerleşimi (metin/bar) tasarla.
- Game Mode'dan Create Widget + Add to Viewport ile ekrana ekle.

⚠ Kısıtlar

- Can değerini widget içinde her Tick'te sorgulamak yerine, HealthComponent'in delegate'ine bağlanmayı tercih et.

✓ Beklenen Sonuç

- Tank hasar aldıkça ekrandaki can göstergesi gerçek zamanlı güncelleniyor.

BÖLÜM II — BONUS (İSTEĞE BAĞLI)

Bonus, zorunlu kısım tam ve eksiksiz çalışır durumda olmadan başlanmamalıdır — tıpkı 42'deki mantık gibi: önce sağlam temel, sonra süsleme.

BONUS · +35 XP (zorunlu %100 tamamlanınca açılır)

Görsel/İşitsel Cila

İlgili dersler: Spawning Particle Systems · Playing Sounds · Camera Shake

Amaç — Vuruş ve çarpışma anlarına parçacık efekti, ses ve kamera sarsıntısı ekleyerek oyuna “his” kazandırmak.

► Yapılacaklar

- Mermi çarpışmasında bir parçacık efekti spawn et.
- Aynı anda bir vuruş sesi çal.
- Oyuncu hasar aldığı anda kısa bir Camera Shake tetikle.

⚠ Kısıtlar

- Efektleri gameplay mantığının (hasar hesaplama) içine karıştırma — ayrı, tepkisel bir katman olarak ekle.

✓ Beklenen Sonuç

- Her vuruşta görsel + işitsel + kamera geri bildirimi aynı anda tetikleniyor.

TESLİM VE ÖZ-DEĞERLENDİRME

Bu bölüm sonunda tam bir oyun döngüsüne (başla → oyna → bitir → yeniden başla) sahip olmalısın. Health Component'i composition ile yazma deneyimin, bir sonraki bölümde (Shooter Sam) hem oyuncu hem AI düşman için tekrar kullanılacak — orada bu deseni değiştirmeden aynen taşıyabilmen beklenir.